6.02 DLLBasics

6.02 DLL

Dynamic Libraries

A **Dynamic Link Library** (DLL) is a library that is loaded separately from the program that uses it. The advantage of DLLs is that you can load one DLL into memory, and multiple programs can use it. This decreases the size of programs.

Static Libraries

A **static library** is linked directly to/with the program that uses it. A library that is statically linked cannot be used by multiple programs at once. The library is part of the program it is used in. This makes the program bigger because of the extra code.

Imports vs Exports

- **Import** - Something brought in from an external source. To use a function from a library you import that function from the library.
- **Export** - Something exposed to the outside so other sources can import it. You're able to access a function from a DLL because the DLL has exported that function. Because it's exported, your program can import it.

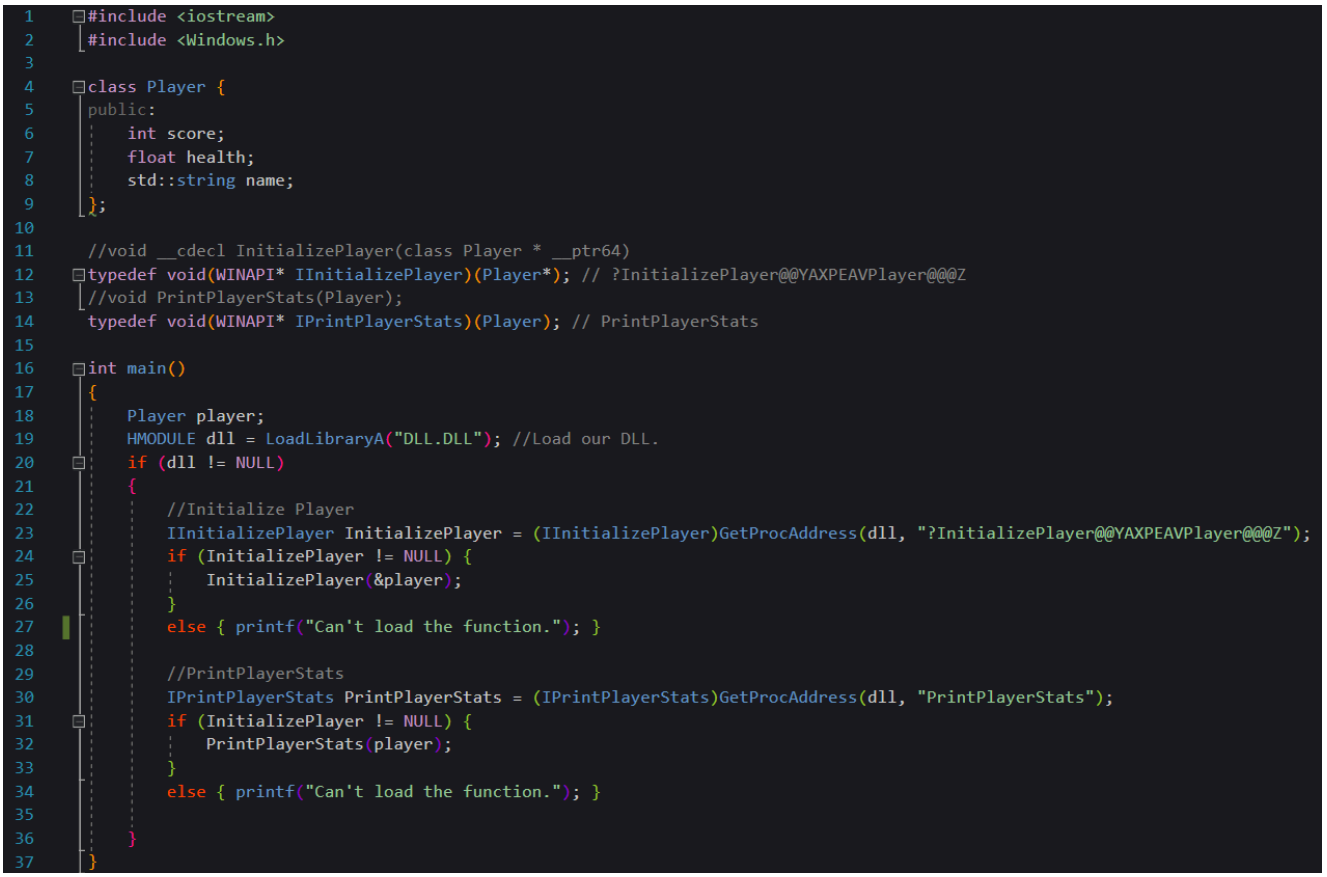
<- Previous Lesson
Next Lesson ->

Chapter Home

6.09 ImplementingPlayer

OPTIONAL: 6.09 Implementing `Player`

Now that we've reversed the `Player` class, let's write our own program that makes a `Player` class and uses the functions related to the class.



```
#include <iostream>
#include <Windows.h>

class Player {
public:
    int score;
    float health;
    std::string name;
};

 //void __cdecl InitializePlayer(class Player * __ptr64)
typedef void(WINAPI* IInitializePlayer)(Player*); // ?InitializePlayer@@YAXPEAVPlayer@@@Z
//void PrintPlayerStats(Player);
 typedef void(WINAPI* IPrintPlayerStats)(Player); // PrintPlayerStats

int main()
{
    Player player;
    HMODULE dll = LoadLibraryA("DLL.DLL"); //Load our DLL.
    if (dll != NULL)
    {
        //Initialize Player
        IInitializePlayer InitializePlayer = (IInitializePlayer)GetProcAddress(dll, "?InitializePlayer@@YAXPEAVPlayer@@@Z");
        if (InitializePlayer != NULL) {
            InitializePlayer(&player);
        }
        else { printf("Can't load the function."); }

        //PrintPlayerStats
        IPrintPlayerStats PrintPlayerStats = (IPrintPlayerStats)GetProcAddress(dll, "PrintPlayerStats");
        if (InitializePlayer != NULL) {
            PrintPlayerStats(player);
        }
        else { printf("Can't load the function."); }

    }
}
```

The player's name doesn't have to be a `std::string`, it can probably be a `const* char` as well.

Copy/Paste Code

```
#include <iostream>
#include <Windows.h>

class Player {
public:
    int score;
    float health;
    std::string name;
};

//void __cdecl InitializePlayer(class Player * __ptr64)
typedef void(WINAPI* IInitializePlayer)(Player*); // ?
InitializePlayer@@YAXPEAVPlayer@@@Z
//void PrintPlayerStats(Player);
typedef void(WINAPI* IPrintPlayerStats)(Player); // PrintPlayerStats

int main()
{
    Player player;
    HMODULE dll = LoadLibraryA("DLL.DLL"); //Load our DLL.
```

```
    if (dll != NULL)
    {
        //Initialize Player
        IInitializePlayer InitializePlayer = (IInitializePlayer)GetProcAddress(dll,
"?InitializePlayer@@YAXPEAVPlayer@@@Z");
        if (InitializePlayer != NULL) {
            InitializePlayer(&player);
        }
        else { printf("Can't load the function."); }

        //PrintPlayerStats
        IPrintPlayerStats PrintPlayerStats = (IPrintPlayerStats)GetProcAddress(dll,
"PrintPlayerStats");
        if (InitializePlayer != NULL) {
            PrintPlayerStats(player);
        }
        else { printf("Can't load the function."); }
    }
}
```

[<- Previous Lesson]
[Next Lesson ->]

[Chapter Home]